

TREE

자료구조 스터디 Week6

학생회

 Made with Gamma

INDEX

01

THEORY

02

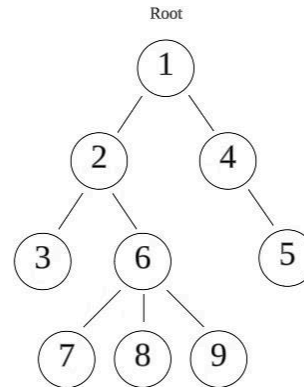
IMPLEMENTATION

03

PROBLEM SOLVING

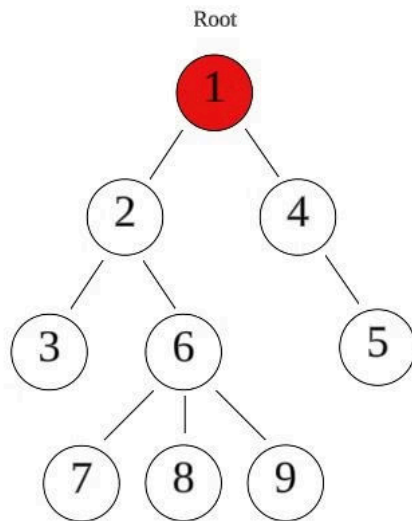
트리

트리(Tree)는 계층적인 구조를 가진 자료구조로서, 노드(Node)와 그 노드들을 연결하는 간선(Edge)으로 구성됩니다. 루트 노드에서부터 시작하여 간선을 따라 하위 노드로 이동할 수 있습니다.



Root

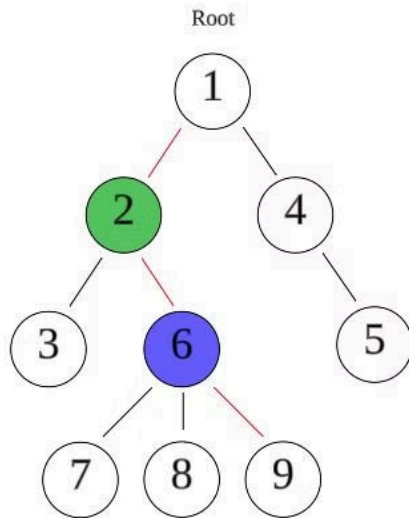
Root - 부모를 갖지 않는 최상위 노드



- 현재 트리의 Root 는 1번 노드

Parent

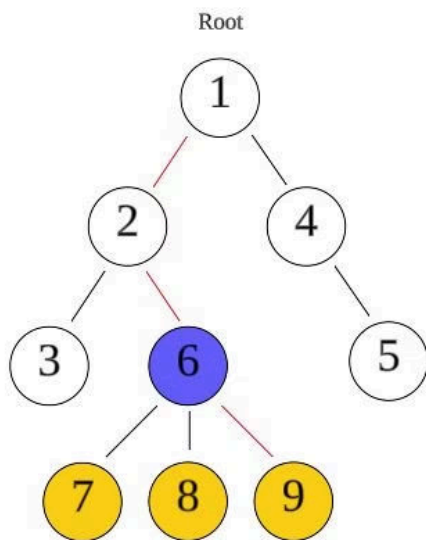
Parent - 특정 노드의 부모 노드



- 6번 노드의 Parent는 2번 노드

Child

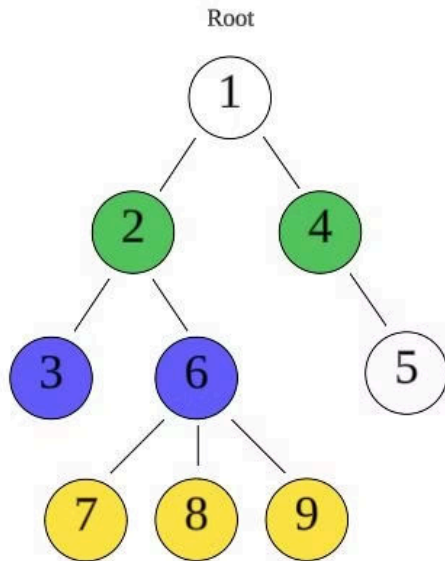
Child - 특정 노드의 자식 노드



- 6번 노드의 Child는 7번, 8번, 9번 노드

Sibling

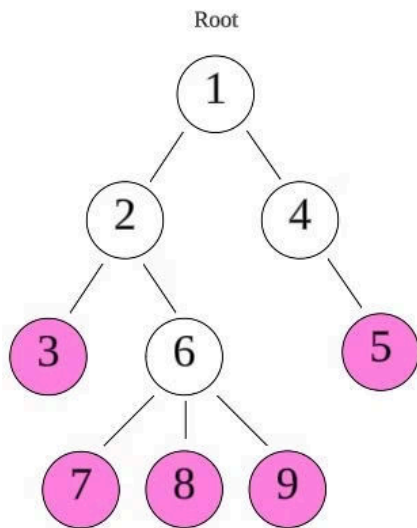
Sibling - 부모가 같은 노드들



- 2번, 4번 노드는 Sibling
- 3번, 6번 노드는 Sibling
- 7번, 8번, 9번 노드는 Sibling

Leaf

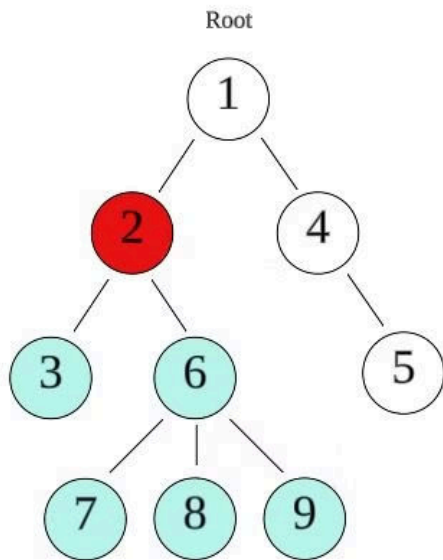
Leaf - 자식 노드가 없는 노드



- 3번, 7번, 8번, 9번, 5번 노드는 Leaf

Subtree

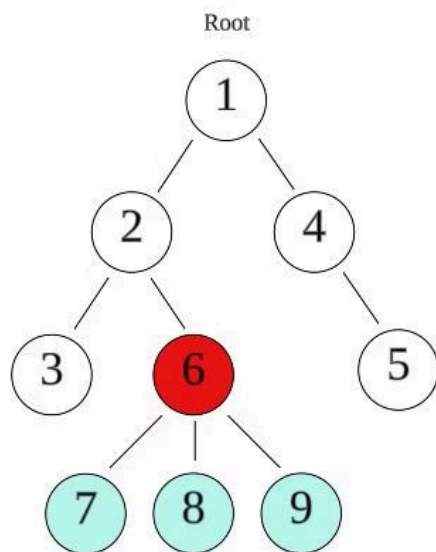
Subtree - 특정 노드와 해당 노드의 하위 노드들로 구성된 트리



- 2번 노드가 Root인 Subtree

Subtree

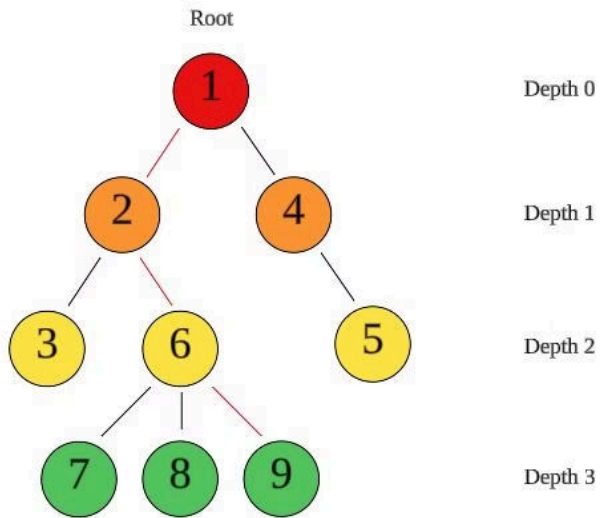
Subtree - 특정 노드와 해당 노드의 하위 노드들로 구성된 트리



- 6번 노드가 Root인 Subtree

Depth

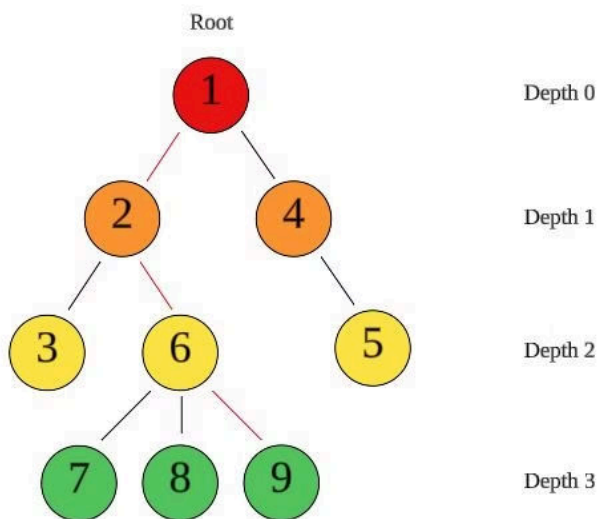
Depth - 루트에서 자신까지 가는 경로의 길이



- Sibling 노드의 Depth는 모두 같음
- 의도적으로 헛갈리게 그린 그림 주의!

Height

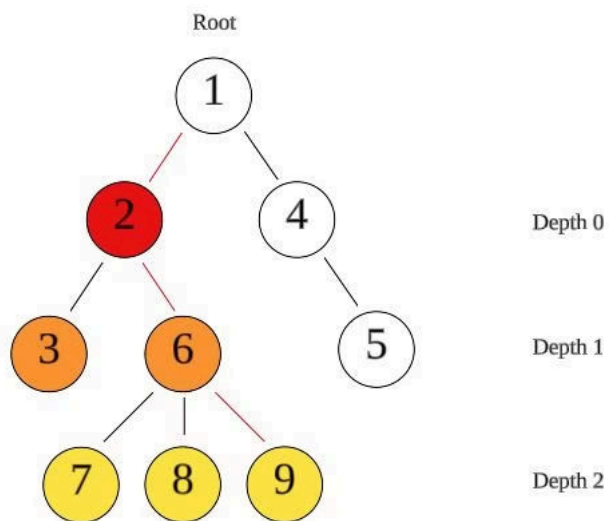
Height - 해당 트리의 모든 노드의 깊이 중 최댓값



- 1번 노드가 Root인 트리의 Height: 3

Height

Height - 해당 트리의 모든 노드의 깊이 중 최댓값



- 2번 노드가 Root인 서브트리의 Height: 2

IMPLEMENTATION

트리의 기본 연산

1 Find

특정 노드 탐색

2 Insert

트리에 노드 삽입

3 Delete

트리의 노드 삭제

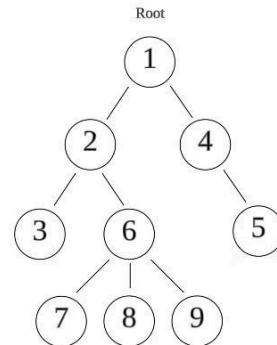
4 Parent

특정 노드의 부모 리턴

5 Child

특정 노드의 자식 리턴

6



노드 클래스

```
class Node {  
private:  
    int value;  
    Node* parent;  
    vector<Node*> child_list;  
  
public:  
    Node(int value, Node* parent) {  
        this->value = value;  
        this->parent = parent;  
    }  
  
    friend class Tree;  
};
```

- 값, 부모 노드, 자식 노드들
- 값이 value, 부모 노드가 parent인 노드 생성
- friend class → 트리에서 노드 편하게 사용

멤버 변수

```
class Tree {  
private:  
    Node* root;  
    vector<Node*> node_list;
```

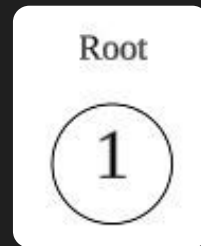
root - 트리의 루트

node_list - 트리의 전체 노드 리스트

생성자

```
public:  
    Tree() {  
        root = new Node(1, nullptr);  
        node_list.push_back(root);  
    }
```

초기 상태 - 트리에 1번 노드가 존재



Find

```
int find(int value, vector<Node*>& list) {  
    for (int i = 0; i < list.size(); i++) {  
        if (list[i]->value == value) {  
            return i;  
        }  
    }  
  
    return -1;  
}
```

- 인자 - 탐색할 노드의 번호, 탐색할 리스트
- 리스트에서 탐색 노드의 번호가 있다면 리스트에서의 index를, 없다면 -1 리턴
- node_list[index], cur_node->child_list[index]로 해당 노드 사용 가능

Insert

```
void insert_node(int par_value, int value) {
    if (find(value, node_list) != -1) {
        cout << "-1\n"; return;
    }

    int par_idx = find(par_value, node_list);
    if (par_idx == -1) {
        cout << "-1\n"; return;
    }

    Node* par_node = node_list[par_idx];
    Node* new_node = new Node(value, par_node);

    par_node->child_list.push_back(new_node);
    node_list.push_back(new_node);
}
```

- 인자 - 부모 노드의 번호, 새 노드의 값
- 삽입할 노드의 번호가 이미 있거나, 입력받은 부모 노드의 번호가 없다면 예외처리
- index로 부모 노드 사용, 부모 노드에 자식 추가
- 삽입된 노드는 `node_list`에도 추가해줘야함!

Delete

```
void delete_node(int value) {
    int idx = find(value, node_list);
    if (idx == -1) {
        cout << "-1\n"; return;
    }

    Node* del_node = node_list[idx];
    if (del_node == root)
        return;

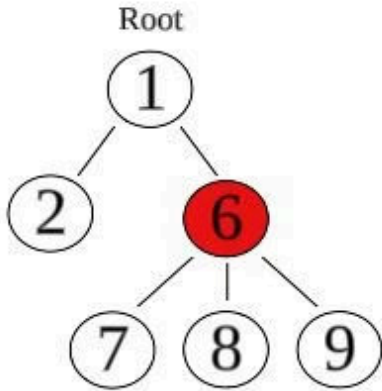
    Node* par_node = del_node->parent;

    for (int i = 0; i < del_node->child_list.size(); i++) {
        par_node->child_list.push_back(del_node->child_list[i]);
        del_node->child_list[i]->parent = par_node;
    }

    vector<Node*>& child = par_node->child_list;
    child.erase(child.begin() + find(value, child));
    node_list.erase(node_list.begin() + idx);
    delete del_node;
}
```

- 인자 - 삭제할 노드의 값
- 삭제할 노드의 번호가 없거나, 삭제할 노드가 Root라면 예외처리
- index로 del_node 사용
- del_node->parent로 부모 노드 사용
- del_node의 자식들을 del_node 부모 노드 child_list에 추가, 그 자식들의 부모를 변경
- 부모 노드의 child_list에서 del_node 삭제
- 삭제된 노드는 node_list에서도 삭제해줘야함!

delete_node(6)

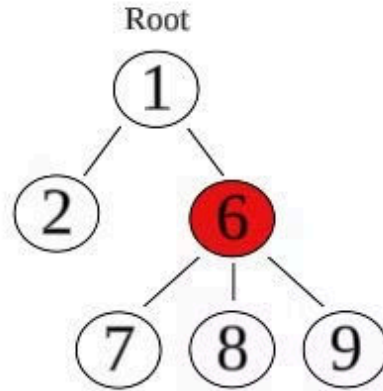


child_list(1)

2	6
---	---

child_list(6)

7	8	9
---	---	---

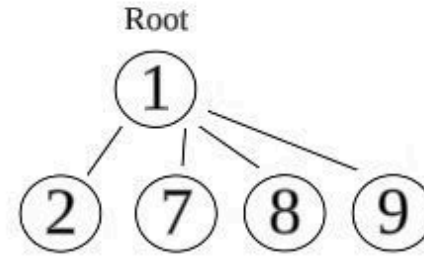


child_list(1)

2	6	7	8	9
---	---	---	---	---

child_list(6)

7	8	9
---	---	---



child_list(1)

2	7	8	9
---	---	---	---

child_list(6)

Parent

```
void print_parent(int value) {  
    int idx = find(value, node_list);  
  
    if (idx == -1) {  
        cout << "-1\n"; return;  
    }  
  
    Node* cur_node = node_list[idx];  
    cout << cur_node->parent->value << '\n';  
}
```

- 문제에 제시된 예외처리
- 특정 노드의 부모 노드 출력

Child

```
void print_child(int value) {
    int idx = find(value, node_list);
    if (idx == -1) {
        cout << "-1\n"; return;
    }

    vector<Node*>& child = node_list[idx]->child_list;
    if (child.empty()) {
        cout << "-1\n"; return;
    }

    for (int i = 0; i < child.size(); i++) {
        cout << child[i]->value << " ";
    }
    cout << '\n';
}
```

- 문제에 제시된 예외처리
- 특정 노드의 자식 노드들 출력

PROBLEM SOLVING

**THANK'S FOR
WATCHING**